

Problem: Minimum Spanning Tree using Kruskal's Algorithm

Objective

To implement Kruskal's Algorithm in C to find the Minimum Spanning Tree (MST) of a weighted, undirected graph, and to compare its execution time against Prim's Algorithm.

Code

```
#include <stdio.h>
#include <stdlib.h>

// -----
// Edge structure
// -----
typedef struct {
    int src, dest, weight;
} Edge;

// -----
// Disjoint Set (Union-Find) with path compression + union by rank
// -----
int find(int parent[], int i) {
    if (parent[i] != i)
        parent[i] = find(parent, parent[i]); // Path compression
    return parent[i];
}

void unionSets(int parent[], int rank[], int x, int y) {
    int rootX = find(parent, x);
    int rootY = find(parent, y);

    if (rootX == rootY) return;

    if (rank[rootX] < rank[rootY])
        parent[rootX] = rootY;
    else if (rank[rootX] > rank[rootY])
        parent[rootY] = rootX;
    else {
        parent[rootY] = rootX;
        rank[rootX]++;
    }
}

// -----
// Comparison function for qsort (sort edges by weight)
// -----
int compareEdges(const void *a, const void *b) {
    Edge *e1 = (Edge *)a;
    Edge *e2 = (Edge *)b;
    return e1->weight - e2->weight;
}

// -----
// Kruskal's Algorithm
// -----
void kruskalMST(Edge edges[], int V, int E) {
```

```

// Step 1: Sort all edges by increasing weight
qsort(edges, E, sizeof(Edge), compareEdges);

int *parent = (int *)malloc(V * sizeof(int));
int *rank_ = (int *)malloc(V * sizeof(int));
for (int v = 0; v < V; v++) {
    parent[v] = v;
    rank_[v] = 0;
}

Edge *result = (Edge *)malloc((V - 1) * sizeof(Edge));
int e = 0; // Edges included in MST so far
int i = 0; // Index into sorted edges
int totalWeight = 0;

printf("Edges considered (in increasing order of weight):\n");
while (e < V - 1 && i < E) {
    Edge nextEdge = edges[i++];

    int x = find(parent, nextEdge.src);
    int y = find(parent, nextEdge.dest);

    if (x != y) {
        result[e++] = nextEdge;
        unionSets(parent, rank_, x, y);
        printf("  Accepted: %d -- %d (weight %d)\n",
            nextEdge.src, nextEdge.dest, nextEdge.weight);
        totalWeight += nextEdge.weight;
    } else {
        printf("  Rejected: %d -- %d (weight %d) -> would form a cycle\n",
            nextEdge.src, nextEdge.dest, nextEdge.weight);
    }
}

printf("\nMinimum Spanning Tree (Kruskal's Algorithm)\n");
printf("-----\n");
for (int j = 0; j < e; j++) {
    printf("  %d -- %d   weight = %d\n", result[j].src, result[j].dest,
result[j].weight);
}
printf("-----\n");
printf("Total edges in MST : %d\n", e);
printf("Total weight of MST: %d\n", totalWeight);

free(parent);
free(rank_);
free(result);
}

int main() {
    // Sample weighted, undirected graph: 7 vertices (0-6), 11 edges
    int V = 7;
    int E = 11;
    Edge edges[] = {
        {0, 1, 7},
        {0, 3, 5},
        {1, 2, 8},
        {1, 3, 9},
        {1, 4, 7},

```

```

        {2, 4, 5},
        {3, 4, 15},
        {3, 5, 6},
        {4, 5, 8},
        {4, 6, 9},
        {5, 6, 11}
    };

    printf("=====\n");
    printf("    KRUSKAL'S ALGORITHM - MINIMUM SPANNING TREE\n");
    printf("=====\n");
    printf("Graph: %d vertices, %d edges\n\n", V, E);

    kruskalMST(edges, V, E);

    return 0;
}

```

Output

```

=====
    KRUSKAL'S ALGORITHM - MINIMUM SPANNING TREE
=====
Graph: 7 vertices, 11 edges

Edges considered (in increasing order of weight):
Accepted: 0 -- 3 (weight 5)
Accepted: 2 -- 4 (weight 5)
Accepted: 3 -- 5 (weight 6)
Accepted: 0 -- 1 (weight 7)
Accepted: 1 -- 4 (weight 7)
Rejected: 1 -- 2 (weight 8) -> would form a cycle
Rejected: 4 -- 5 (weight 8) -> would form a cycle
Rejected: 1 -- 3 (weight 9) -> would form a cycle
Accepted: 4 -- 6 (weight 9)

Minimum Spanning Tree (Kruskal's Algorithm)
-----
    0 -- 3   weight = 5
    2 -- 4   weight = 5
    3 -- 5   weight = 6
    0 -- 1   weight = 7
    1 -- 4   weight = 7
    4 -- 6   weight = 9
-----
Total edges in MST : 6
Total weight of MST: 39

```

How Kruskal's Algorithm Works

1. Sort all edges of the graph in non-decreasing order of their weight.

2. Initialize a Disjoint Set (Union-Find) structure with each vertex in its own set.
3. Walk through the sorted edges one at a time. For each edge, check whether its two endpoints belong to different sets (using find()).
4. If they belong to different sets, include the edge in the MST and merge (union) the two sets. If they belong to the same set, discard the edge — adding it would create a cycle.
5. Stop once the MST contains $(V - 1)$ edges, where V is the number of vertices.

Summary

Aspect	Kruskal's Algorithm
Type	Greedy, Edge-based
Cycle Detection	Union-Find (Disjoint Set)
Sorting Required	Yes — all edges by weight
Time Complexity	$O(E \log E)$
Space Complexity	$O(V + E)$
Works Well On	Sparse graphs, edge-list representations